ECE 526L Spring 2025 Lab 6

Design and Verification of a Parameterized Multiplexer



Muhammad Umar
CSUN ID: 204533244

Instructor: Prof. Orod Haghighiara

I hereby attest that this lab report is entirely my own work. I have not copied either code or text from anyone, nor have I allowed or will I allow anyone to copy my work.

Name (printed): **Muhammad Umar**

Name (signed):  Date: 04/04/2025

INTRODUCTION

This lab focuses on the design and simulation of a parameterized multiplexer (MUX) using SystemVerilog. The primary objective is to implement a synthesizable, scalable multiplexer that supports variable data widths and correctly resolves outputs under all conditions, including unknown (**x**) select values. The design must avoid any explicit or implicit comparison to x on the **sel** signal, following the synthesis friendly practices.

The testbench verifies the functionality of four instances of the multiplexer as required, using various parameter redefinition methods which includes using **default** parameter, **defparam**, **localparam** and module parameter list. It ensures complete test coverage for all possible select input values including **x**, and verifies that all output bits toggle between logic 0 and 1 at least once and also checks various conditions which can be seen in Appendix of log file **lab6.log**.

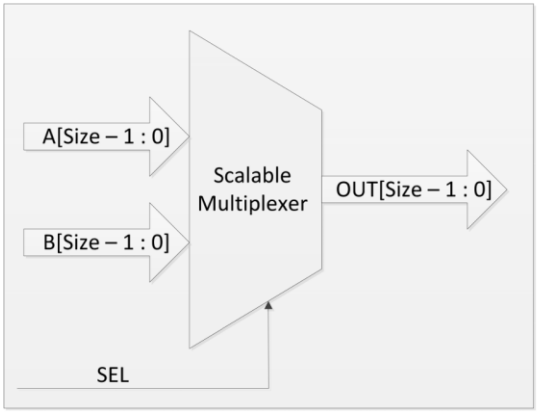


Figure 6.1 Schematic of Parametrized MUX

This design is purely combinational and does not require a clock signal, making it ideal for use in data routing and selection logic where synchronous timing is not essential.

METHODOLOGY AND TEST PLAN

1. Multiplexer Module (mux.sv):

The core component of the lab is a parameterized multiplexer that supports scalable data widths defined by the **SIZE** parameter. If not explicitly defined during instantiation, it defaults to 1-bit. The module takes two SIZE bit inputs (**A** and **B**) and a single bit **sel** input. It produces an output of the same width. The logic is combinational and thus does not require a clock signal so we will not be using any clock then in our design.

```
module mux #(
  parameter SIZE = 1
)(
  input  logic [SIZE-1:0] A,
  input  logic [SIZE-1:0] B,
  input  logic            sel,
  output logic [SIZE-1:0] out
);

always_comb begin
  for (int i = 0; i < SIZE; i++) begin
    case (sel)
      1'b0: out[i] = A[i];
      1'b1: out[i] = B[i];
      default: begin
        // sel = x or z
        if (A[i] === B[i])
          out[i] = A[i];
        else
          out[i] = 1'bx;
      end
    endcase
  end
end

endmodule
```

Figure 6.2 MUX Design Code

The behavior of the design is defined as:

- When **sel** = **0**: output = **A**
- When **sel** = **1**: output = **B**
- When **sel** = **x**: resolve each bit individually:
  - If A[i] == B[i]: output that bit

- else: output x

We have used **Case** Statement for the selection of right output and also used for loop and iterated it till the **SIZE** and compare every bit of input **A** and **B** and then stored it in register **out.**

This approach avoids illegal synthesis operations like comparing sel to x as instructed to not use any comparison to **x**.

2. Testbench Module (mux_tb.sv):

The testbench module **mux_tb.sv** is responsible for verifying the correctness and of the scalable multiplexer design under various conditions as required. Four different instances of the MUX are instantiated with widths of **1, 4, 5,** and **6** bits. Each instance demonstrates a different method of parameter redefinition, with one using the **default** width and others modified via **defparam**, **inline** parameter override, and **localparam**.

The input vectors **A**, **B**, and control signal **sel** are shared across all instances to allow parallel verification. This ensures that each MUX instance is tested simultaneously under identical input conditions.

To validate the functional behavior, a series of test cases were implemented:

- **Case 1:** When sel = 0, all outputs are expected to match input A.
- **Case 2:** When sel = 1, outputs should match input B.
- **Case 3:** When sel = x and inputs A and B are equal, the output should resolve to A with no undefined bits.
- **Case 4:** When sel = x and inputs A and B are completely different, the output should resolve to all **x** bits.
- **Case 5:** When sel = x and some bits of A and B are equal while others are not, the output should show valid values for matching bits and **x** for conflicting bits.
- **Case 6:** Inputs are configured to ensure every output bit transitions from 0 to 1, showing complete toggling and full coverage.

```
********************************************************************************
Starting Parameterized MUX Testbench
********************************************************************************

********************************************************************************
CASE 1: sel = 0 → output should be A

sel=0 | A=101010 B=010101 | out1=0 out4=1010 out5=01010 out6=101010

********************************************************************************
CASE 2: sel = 1 → output should be B

sel=1 | A=111100 B=000011 | out1=1 out4=0011 out5=00011 out6=000011

********************************************************************************
CASE 3: sel = x, A == B → output = A (no x)

sel=x, A==B | A=110011 B=110011 | out1=1 out4=0011 out5=10011 out6=110011

********************************************************************************
CASE 4: sel = x, A != B → output = x (conflicts)

sel=x, A!=B | A=111000 B=000111 | out1=x out4=xxxx out5=xxxxx out6=xxxxxx

********************************************************************************
CASE 5: sel = x, A and B share some bits

sel=x, mixed A/B | A=101011 B=101101 | out1=1 out4=1xx1 out5=01xx1 out6=101xx1

********************************************************************************
CASE 6: Toggle every bit in output to both 0 and 1
Making sure no bits are stuck

All output bits to 0: A=000000 B=111111 | out1=0 out4=0000 out5=00000 out6=000000
All output bits to 1: A=111111 B=000000 | out1=1 out4=1111 out5=11111 out6=111111

********************************************************************************
Mux Testbench Finished.
********************************************************************************

$finish called from file "mux_tb.sv", line 136.
$finish at simulation time                   35
           V C S   S i m u l a t i o n   R e p o r t
```

Figure 6.3Simulation of MUX Design

The testbench demonstrates all three methods for redefining parameters across four instances:

- **Default Instantiation**: The first instance uses the default width of 1 bit.
- **Defparam Override**: The second instance (4-bit) modifies the SIZE parameter using the **defparam** keyword.

- **Inline Parameter Override**: The third instance uses the #(.SIZE(5)) style for 5-bit width.
- **Localparam Assignment**: The final instance uses a localparam to set the width to 6 bits.

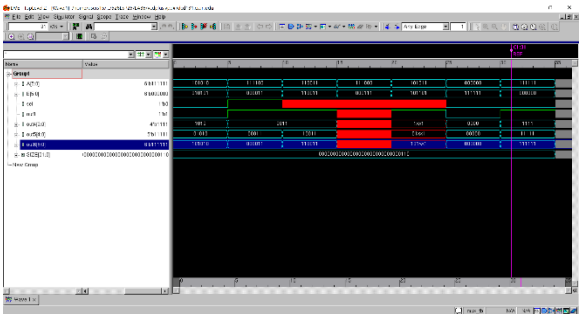


Figure 6.4 Waveform of MUX Design

All instances operate simultaneously under the same input conditions as we can see in the aforementioned waveform which cannot be seen clearly here, although it will be attached upon with submission of lab files.

ANALYSIS OF RESULTS:

The simulation results matched the expected behavior in all test cases. Below is a summary of observed results in a table and tested every corner case as required in lab6 instruction manual:

TABLE I Test Cases Covered in Testbench

Case	Description	Observed Behavior
1	sel = 0	out = A
2	sel = 1	out = B
3	sel = x, A == B	out = A
4	sel = x, A != B	out = x (all)
5	sel = x, some A[i] == B[i]	mix of bits/x
6	Bit toggle coverage (0 → 1)	verified for all outputs

CONCLUSION

This lab demonstrated the complete design and verification flow of a parameterized and synthesizable multiplexer in SystemVerilog. The MUX design was scalable via a configurable SIZE parameter and behaved correctly under all tested conditions, including when the select signal was unknown (x). The implementation also includes bitwise resolution logic that safely handled undefined conditions without violating synthesis constraints.

The use of three distinct methods for parameter redefinition demonstrated versatility in module instantiation. Each test case confirmed the multiplexer's correct operation by comparing expected outputs against simulation results. The waveform data further verified full bit level activity and correctness.

APPENDIX

Appendix A: mux.sv

```
/*****************************************************************************
***                                                                        ***
*** EE 526 L Experiment #6            Muhammad Umar, Spring, 2025          ***
***                                                                        ***
*** Experiment #6 Design and Verification of Parametrized MUX              ***
***                                                                        ***
*** Filename: mux.sv          Created by Muhammad Umar, 04/04/2025         ***
***                                                                        ***
*** This module implements a logic circuit that selects                    ***
*** between two SIZE-bit inputs (A and B) using a 1-bit select signal      ***
*** (sel).  It is a purely combinational design that resolves unknown      ***
*** select conditions at the bit level without requiring any clock.        ***
*****************************************************************************/
  module mux #(
    parameter SIZE = 1
  )(
    input  logic [SIZE-1:0] A,
    input  logic [SIZE-1:0] B,
    input  logic            sel,
    output logic [SIZE-1:0] out
  );

  always_comb begin
    for (int i = 0; i < SIZE; i++) begin
      case (sel)
        1'b0: out[i] = A[i];
        1'b1: out[i] = B[i];
        default: begin
          // sel = x or z
          if (A[i] === B[i])
            out[i] = A[i];
          else
            out[i] = 1'bx;
        end
      endcase
    end
  end

  endmodule
```

Appendix B: mux_tb.sv

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #6            Muhammad Umar, Spring, 2025      ***
***                                                                    ***
*** Experiment #6 Design and Verification of Parametrized MUX          ***
***                                                                    ***
*** Filename: mux_tb.sv             Created by Muhammad Umar, 04/04/2025 ***
***                                                                    ***
*** This testbench module verify the Parametrized Mux design according   ***
*** to the cases defined in lab instruction                           ***
*************************************************************************/

module mux_tb;

  // Taking 6bit reg to cover all the cases
  logic [5:0] A, B;
  logic       sel;

  // Outputs from each mux instances according to their respective
names
  logic out1;
  logic [3:0] out4;
  logic [4:0] out5;
  logic [5:0] out6;

  // Instance 1: default parameter 1-bit width
  mux m1 (
    .A(A[0]),
    .B(B[0]),
    .sel(sel),
    .out(out1)
  );

  // Instance 2: override via defparam
  mux m2 (
    .A(A[3:0]),
    .B(B[3:0]),
    .sel(sel),
    .out(out4)
  );
  defparam m2.SIZE = 4;

  // Instance 3: override via module parameter list
  mux #(.SIZE(5)) m3 (
```

```
    .A(A[4:0]),
    .B(B[4:0]),
    .sel(sel),
    .out(out5)
  );

  // Instance 4: override via localparam
  localparam SIZE = 6;
  mux #(SIZE) m4 (
    .A(A),
    .B(B),
    .sel(sel),
    .out(out6)
  );

// Test sequence
  initial begin
    $vcdpluson;


$display("\n*****************************************************
**********************************");
    $display("Starting Parameterized MUX Testbench");

$display("*****************************************************
**********************************\n");

    // CASE 1: sel = 0 → output = A

$display("*****************************************************
**********************************");
    $display("CASE 1: sel = 0 → output should be A");
    $display("\n");

    sel = 1'b0;
    A   = 6'b101010;
    B   = 6'b010101;
    #5;
    $display("sel=0 | A=%b B=%b | out1=%b out4=%b out5=%b out6=%b",
A, B, out1, out4, out5, out6);

    // CASE 2: sel = 1 → output = B
```

```
$display("\n*************************************************************
***********************************");
    $display("CASE 2: sel = 1 → output should be B");
    $display("\n");

    sel = 1'b1;
    A   = 6'b111100;
    B   = 6'b000011;
    #5;
    $display("sel=1 | A=%b B=%b | out1=%b out4=%b out5=%b out6=%b",
A, B, out1, out4, out5, out6);

    // CASE 3: sel = x, A == B → output = A (no x expected)

$display("\n*************************************************************
***********************************");
    $display("CASE 3: sel = x, A == B → output = A (no x)");
    $display("\n");

    sel = 1'bx;
    A   = 6'b110011;
    B   = 6'b110011;
    #5;
    $display("sel=x, A==B | A=%b B=%b | out1=%b out4=%b out5=%b
out6=%b", A, B, out1, out4, out5, out6);

    // CASE 4: sel = x, A != B → output = x (all bits conflict)

$display("\n*************************************************************
***********************************");
    $display("CASE 4: sel = x, A != B → output = x (conflicts)");
    $display("\n");

    sel = 1'bx;
    A   = 6'b111000;
    B   = 6'b000111;
    #5;
    $display("sel=x, A!=B | A=%b B=%b | out1=%b out4=%b out5=%b
out6=%b", A, B, out1, out4, out5, out6);

    // CASE 5: sel = x, A and B share some bits (mixed match)
```

```
$display("\n************************************************************
**********************************");
    $display("CASE 5: sel = x, A and B share some bits");
    $display("\n");

    sel = 1'bx;
    A   = 6'b101011;
    B   = 6'b101101;
    #5;
    $display("sel=x, mixed A/B | A=%b B=%b | out1=%b out4=%b out5=%b
out6=%b", A, B, out1, out4, out5, out6);

    // EXTRA: Confirm all bits go to 0 and 1

$display("\n************************************************************
**********************************");
    $display("CASE 6: Toggle every bit in output to both 0 and 1");
    $display("Making sure no bits are stuck");
    $display("\n");

    // Each bit goes to 0
    sel = 1'b0;
    A   = 6'b000000;
    B   = 6'b111111;
    #5;
    $display("All output bits to 0: A=%b B=%b | out1=%b out4=%b
out5=%b out6=%b", A, B, out1, out4, out5, out6);

    // Each bit goes to 1
    sel = 1'b0;
    A   = 6'b111111;
    B   = 6'b000000;
    #5;
    $display("All output bits to 1: A=%b B=%b | out1=%b out4=%b
out5=%b out6=%b", A, B, out1, out4, out5, out6);


$display("\n************************************************************
**********************************");
    $display("Mux Testbench Finished.");

$display("************************************************************
**********************************\n");
```

```
    $finish;
  end

endmodule
```

Appendix C: lab6.log

```
Command: /home/users15/mu832615/526LSP25/LAB6/./simv -l lab6.log
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP1_Full64; Runtime version U-2023.03-
SP1_Full64;  Apr  5 01:02 2025
VCD+ Writer U-2023.03-SP1_Full64 Copyright (c) 1991-2023 by Synopsys
Inc.

**************************************************************************
************************
Starting Parameterized MUX Testbench
**************************************************************************
************************

**************************************************************************
************************
CASE 1: sel = 0 → output should be A


sel=0 | A=101010 B=010101 | out1=0 out4=1010 out5=01010 out6=101010

**************************************************************************
************************
CASE 2: sel = 1 → output should be B


sel=1 | A=111100 B=000011 | out1=1 out4=0011 out5=00011 out6=000011

**************************************************************************
************************
CASE 3: sel = x, A == B → output = A (no x)


sel=x, A==B | A=110011 B=110011 | out1=1 out4=0011 out5=10011
out6=110011

**************************************************************************
************************
CASE 4: sel = x, A != B → output = x (conflicts)


sel=x, A!=B | A=111000 B=000111 | out1=x out4=xxxx out5=xxxxx
out6=xxxxxx
```

```
********************************************************************************
************************
CASE 5: sel = x, A and B share some bits


sel=x, mixed A/B | A=101011 B=101101 | out1=1 out4=1xx1 out5=01xx1
out6=101xx1

********************************************************************************
************************
CASE 6: Toggle every bit in output to both 0 and 1
Making sure no bits are stuck


All output bits to 0: A=000000 B=111111 | out1=0 out4=0000 out5=00000
out6=000000
All output bits to 1: A=111111 B=000000 | out1=1 out4=1111 out5=11111
out6=111111

********************************************************************************
************************
Mux Testbench Finished.
********************************************************************************
************************

$finish called from file "mux_tb.sv", line 136.
$finish at simulation time                   35
           V C S   S i m u l a t i o n   R e p o r t
Time: 35
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Sat Apr  5 01:02:07 2025
```